

ESP32 Architecture & Embedded C/C++ Concepts

Memory Architecture, C/C++ Memory Mechanics, OOP Principles .

Report Title	ESP32 Architecture & Embedded C/C++ Concepts
Program	IoT Training Program
Date	26/7/2026
Organization	Cyber Robot

Confidential — Internal Training Documentation

Contents

Contents.....	Error! Bookmark not defined.
Goal of This Report.....	3
Section 1: ESP32 Memory Architecture & Layout	4
1. The ESP32 Memory Map	4
DRAM (Data RAM).....	4
IRAM (Instruction RAM)	4
PSRAM (External SPI RAM)	5
2. General MCU Memory Sections.....	5
Where Do These Five Example Variables Live?	6
3. ESP32 Directives & Keywords	7
volatile	7
IRAM_ATTR.....	8
Section 2: Pointers, References & Memory Management.....	10
4. Pointers vs. References (C vs. C++)	10
Why const Reference Beats Pass-by-Value and Pass-by-Pointer	10
5. Structs, Enums, and Unpacking Data	11
C-Style enum vs. C++ enum class (Scoped Enums)	11
Struct Padding & Alignment.....	12
6. Function Pointers & Callbacks.....	13
Section 3: C++ OOP & Access Modifiers in Embedded Systems	14
7. Access Specifiers (public, private, protected)	14
private vs. protected When Inheriting a Base Driver Class.....	14
8. Constructors & Initialization Lists	15
Why Use a Member Initializer List Instead of the Constructor Body	15
9. Cost of C++ Features in Embedded Systems.....	15
Virtual Functions and the vtable.....	15
Dynamic Memory Allocation (new / delete / malloc / free)	17
References	18

Goal of This Report

This report investigates the ESP32 memory architecture, the underlying C/C++ memory mechanics that govern where every variable and function actually lives, core C++ object-oriented programming principles, and basic hardware protection concepts. The material is written to prepare a firmware engineer to move confidently into production driver development and FreeRTOS-based multitasking, where getting memory placement, pointer usage, and class design wrong can cause crashes, memory leaks, or missed real-time deadlines rather than just compiler warnings.

IN PLAIN TERMS

Everything in this report answers one practical question for a firmware developer: “where does this piece of data or code actually live while the chip is running, and what does that location cost me in speed, size, or reliability?”

Section 1: ESP32 Memory Architecture & Layout

1. The ESP32 Memory Map

The ESP32 uses a Harvard architecture, meaning instructions and data travel over separate internal paths so the CPU can fetch the next instruction while it is still reading or writing data from a previous one. Physically, the memory available to a running ESP32 program is split between memory that sits inside the silicon die itself (internal memory) and memory that sits on the module, just outside the chip, connected over SPI (external memory). The diagram below places every memory type discussed in this section on that map.

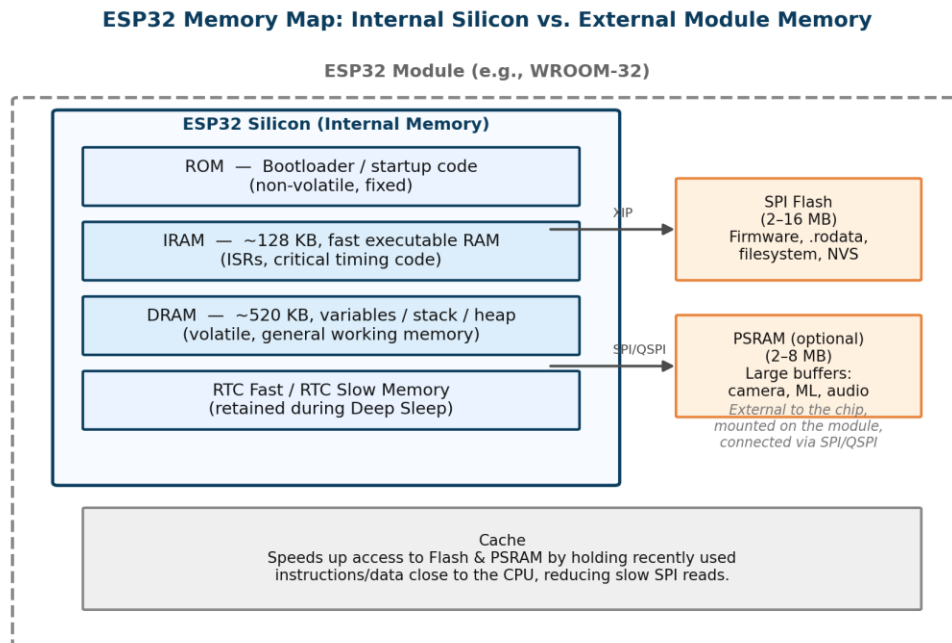


Figure 1.1 — Internal ESP32 silicon memory (ROM, IRAM, DRAM, RTC memories) versus external module memory (SPI Flash, PSRAM).

DRAM (Data RAM)

DRAM is the ESP32's general-purpose internal working memory. On the classic ESP32 it totals roughly 520 KB of fast, volatile SRAM — volatile meaning its contents are lost the instant power is removed. DRAM is where the running program keeps everything it needs moment to moment: global and static variables that were given an initial value or left uninitialized, the call stack that tracks which function called which, and the heap used for dynamically allocated memory such as `malloc()` or `new`. Because it sits on the same die as the CPU core, DRAM access is very fast and has low, predictable latency, which is why it is the default home for almost all runtime data.

- Capacity: roughly 520 KB on the classic ESP32 (varies by variant).
- Speed: very fast, low-latency internal SRAM access.
- Typical use: global/static variables, the stack, and the heap.

IRAM (Instruction RAM)

IRAM is a much smaller pool of fast internal RAM — typically around 128 KB — reserved specifically for executable instructions rather than data. Most firmware does not need it at all, because most functions execute directly from Flash using a mechanism called Execute-In-Place (XIP), backed by an instruction cache. IRAM exists for the narrow set of cases where Flash cannot be trusted to be available: during a Flash erase or write, during an OTA (over-the-air) firmware update, or whenever the instruction cache has been disabled. If an interrupt fires during one of those windows and its handler is not sitting in IRAM, the CPU has no valid instructions to fetch, which can produce an illegal-instruction exception, a watchdog reset, or the well-known ESP32 “Guru Meditation Error.”

- Capacity: roughly 128 KB — far too small to hold an entire application.
- Speed: very low latency, comparable to DRAM, since it is also internal SRAM.
- Typical use: interrupt service routines (ISRs), critical timing code, and any function that must keep running while Flash is temporarily inaccessible.

PSRAM (External SPI RAM)

PSRAM (Pseudo-Static RAM, also called SPIRAM) is optional external memory, typically 2–8 MB, added on WROVER-class modules and connected to the ESP32 over a high-speed SPI/QSPI link rather than being part of the silicon die. It behaves like ordinary volatile RAM from the programmer's point of view, but because every access has to cross the external SPI bus, it is noticeably slower than the internal DRAM sitting directly on the chip. PSRAM earns its cost when a workload simply will not fit in ~520 KB of internal SRAM — camera frame buffers, full-screen display buffers, audio processing buffers, or on-device machine-learning models are the standard examples.

- Capacity: 2–8 MB, far larger than internal DRAM.
- Speed: slower than internal DRAM because every access crosses the external SPI/QSPI bus.
- Typical use: large buffers that cannot fit in internal SRAM — camera frames, display framebuffers, ML models.

WHY THIS MATTERS

Choosing the right memory is a trade-off, not a preference. DRAM and IRAM are fast because they are physically on the die, but there is very little of either. PSRAM is abundant but pays a latency penalty for every access because the data has to travel off-chip over SPI. Flash is the largest and cheapest, but the least suited to code that must run no matter what — which is exactly the gap IRAM_ATTR closes, covered later in this section.

2. General MCU Memory Sections

Underneath any specific chip's memory map, the C/C++ compiler and linker organize every compiled program into a small, standard set of memory sections. These sections are not separate files — they are labelled regions inside the object file (.o) and, after linking, inside the final executable (.elf). Understanding what each section holds explains exactly where a given variable or function will end up once the program is running.

Where Compiled C/C++ Sections Live at Runtime on the ESP32

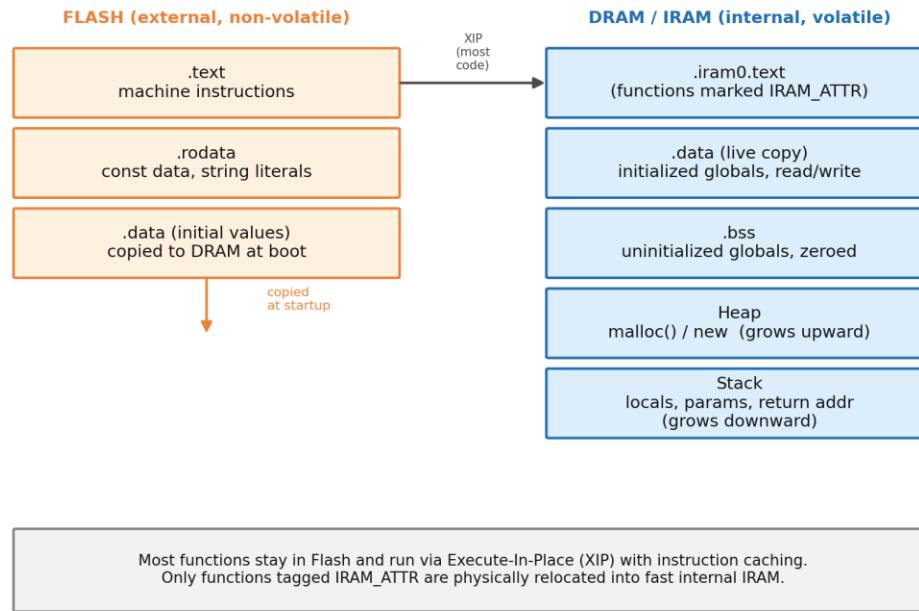


Figure 1.2 — Compiled C/C++ sections mapped onto ESP32 Flash and internal DRAM/IRAM at runtime.

Section	Stores	Access	ESP32 Location
.text	Compiled machine instructions (executable code)	Read-only	Flash (or IRAM for functions tagged IRAM_ATTR)
.rodata	Constants (const) and string literals	Read-only	Flash
.data	Initialized global/static variables	Read/Write	Copied from Flash into DRAM at startup
.bss	Uninitialized global/static variables (auto zero-filled)	Read/Write	DRAM only — never stored in Flash
Stack	Local variables, function parameters, return addresses	Read/Write	DRAM, grows and shrinks automatically per function call
Heap	Dynamically allocated memory (malloc(), new)	Read/Write	DRAM, until free()/delete is called

A natural follow-up question: if DRAM loses everything when power is removed, how does an initialized variable such as `int speed = 100;` still show 100 the moment the program starts? The initial value 100 is stored once in Flash as part of the `.data` section's "template." During startup, before `main()/setup()` runs, the bootloader copies that template out of Flash into the live `.data` region in DRAM. From that point on, the CPU reads and writes only the RAM copy; Flash is never touched again for that variable during normal operation. Uninitialized globals in `.bss` skip this copy step entirely — the startup code simply allocates the RAM and fills it with zeros, which is far cheaper than storing thousands of zero bytes inside Flash itself.

Where Do These Five Example Variables Live?

Declaration	Section	Reasoning
<code>const int x = 5;</code>	<code>.rodata</code>	Constant, read-only, never changes — kept in Flash.
<code>int y = 5; (global)</code>	<code>.data</code>	Global with an explicit initial value — copied into DRAM at boot.
<code>int z; (global)</code>	<code>.bss</code>	Global with no initial value — zero-filled directly in DRAM.
<code>int k = 10; (inside a function)</code>	Stack	Local variable — exists only for the duration of that function call.
Buffer from <code>malloc()</code> / <code>new</code>	Heap (pointer itself is on the Stack if local)	Explicitly, manually allocated memory that persists until freed.

3. ESP32 Directives & Keywords

Modern compilers aggressively optimize code on the assumption that a variable only changes because of instructions the compiler itself can see, and that a function's location in memory does not matter beyond correctness. Both assumptions break down on an interrupt-driven embedded system, which is exactly why C/C++ and ESP-IDF provide `volatile` and `IRAM_ATTR` to override the compiler's default behavior in these two specific situations.

volatile

`volatile` is a C/C++ type qualifier that instructs the compiler: this variable may change unexpectedly, outside the normal flow of the code the compiler can see — always read its actual value from memory rather than assuming a cached copy is still correct. Without `volatile`, the compiler may cache a variable's value inside a CPU register the first time it is read inside a loop, and simply never look at memory again for the rest of that loop, because nothing in the visible code appears to change it.

This is precisely the situation created by an Interrupt Service Routine (ISR): an ISR can fire at literally any point during normal program execution, invisibly to the surrounding code, and update a shared flag. If that flag is not declared `volatile`, the compiler is free to assume it never changes inside a polling loop and may optimize the loop into one that checks a stale register value forever, so the main program never notices the ISR ever ran.

```
volatile bool buttonPressed = false;

void IRAM_ATTR buttonISR()
{
    buttonPressed = true;    // set from inside an interrupt
}

void loop()
{
    if (buttonPressed)      // volatile forces a fresh memory read here
    {
```

```

        buttonPressed = false;
    }
}

```

Specifically, `volatile` prevents four related compiler optimizations: caching the variable's value in a CPU register instead of memory, eliminating what look like “repeated” reads of the same variable, assuming the variable's value never changes across a loop, and freely reordering reads/writes of that variable relative to surrounding code. It is important to be precise about what `volatile` does not do: it does not make an operation atomic, and it provides no thread synchronization or mutual exclusion between FreeRTOS tasks or CPU cores — for that, mutexes, semaphores, or critical sections are still required.

IRAM_ATTR

`IRAM_ATTR` is an ESP32-specific function attribute that instructs the linker to physically place that function's compiled code into Instruction RAM instead of leaving it in Flash. Under normal conditions the CPU executes almost every function directly from Flash using Execute-In-Place (XIP), and this works efficiently because of memory mapping and the instruction cache. The problem is that certain operations — erasing or writing to Flash, an OTA firmware update, or any state that disables the instruction cache — temporarily make Flash unreadable by the CPU. If an interrupt occurs during exactly that window and its handler's instructions are only in Flash, the CPU cannot fetch them.

```

void IRAM_ATTR timerISR()
{
    timerExpired = true;    // this code is now physically copied into IRAM
}

```

Marking a function `IRAM_ATTR` moves that function's machine code out of the normal `.text` section (which resides in Flash) and into a dedicated executable IRAM section, commonly named `.iram0.text` in ESP-IDF. If `IRAM_ATTR` is omitted on a function that genuinely needs it, the ESP32 can experience increased interrupt latency, an illegal instruction exception, a watchdog timeout, or a full system reset the moment that interrupt happens to land during a Flash-write or OTA operation. Because IRAM is only about 128 KB, it should be reserved for interrupt handlers and other extremely time-critical, Flash-independent code — most ordinary functions, including `setup()` and `loop()`, execute perfectly well from Flash and should stay there.

Aspect	<code>volatile</code>	<code>IRAM_ATTR</code>
Type	C/C++ type qualifier	ESP32-specific function attribute
Applies to	Variables	Functions
Purpose	Prevent incorrect compiler optimizations	Place a function's code in IRAM
Controlled by	Compiler	Linker
Typical use	Shared flags, hardware registers, ISR-visible variables	Interrupt handlers, other critical timing code

Aspect	volatile	IRAM_ATTR
If omitted where needed	Stale cached values; main code never observes ISR updates	ISR may fail / crash if Flash is inaccessible when the interrupt fires

Section 2: Pointers, References & Memory Management

4. Pointers vs. References (C vs. C++)

C only has pointers. C++ adds a second way to refer indirectly to an existing variable: the reference. Both let a function or a piece of code work with data it does not own a copy of, but they differ in three practical ways that matter constantly in firmware code: whether they can be “empty,” whether they can be pointed at something else later, and how they behave with arrays and addresses.

Pointer vs. Reference: What Each One Actually Is in Memory

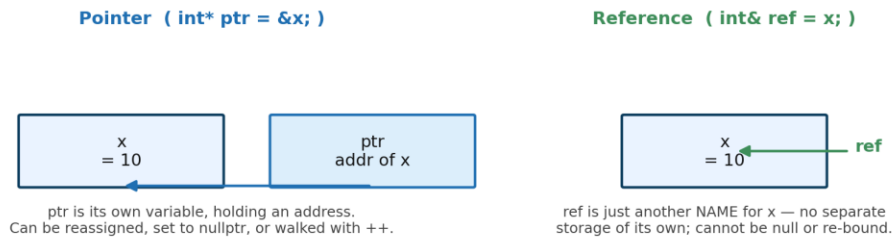


Figure 2.1 — A pointer is its own variable holding an address; a reference is simply another name for an existing variable.

Feature	Pointer (<code>*</code>)	Reference (<code>&</code>)
Nullability	Can be null (nullptr); must be checked before use	Cannot be null — always refers to a real, already-existing object
Reassignment	Can be reassigned to point at a different variable at any time	Bound permanently to one variable at the moment it is declared
Pointer arithmetic	Supports ++ / -- / offset arithmetic; used to walk arrays and buffers	No arithmetic — a reference is not a separate addressable object
Syntax	Declared with <code>*</code> ; must be explicitly dereferenced with <code>*</code> to read/write the target	Declared with <code>&</code> ; used exactly like the original variable, no dereference operator needed
Must be initialized?	No — can be declared and assigned an address later	Yes — must be bound to a variable in the same statement it is declared

Why const Reference Beats Pass-by-Value and Pass-by-Pointer

When a function needs to read a large struct — a sensor-data packet with several fields, for example — there are three ways to hand it to that function, and they are not equivalent in cost:

- Pass by value (`SensorData data`): the entire struct is physically copied onto the stack for every call. For a small int this copy is free; for a multi-field struct it wastes both CPU cycles and stack space on a microcontroller that may only have a few hundred kilobytes of RAM in total.
- Pass by raw pointer (`SensorData* data`): avoids the copy, but the pointer can be null, can be reassigned inside the function, and gives no promise that the function will not modify the caller's data — the caller has to trust the function's implementation, not just its signature.
- Pass by const reference (`const SensorData& data`): also avoids the copy — only a small internal reference/address is passed, not the whole struct — while the `const` keyword lets the compiler enforce, at the function's declaration, that the data cannot be modified, and the reference syntax guarantees the argument can never be null.

```
struct SensorData { float temperature; float humidity; uint32_t timestamp; float raw[8]; };

// Copies the whole struct on every call – wasteful on an MCU
void logReadingByValue(SensorData data) { /* ... */ }

// No copy, but data could be null, and could be modified accidentally
void logReadingByPointer(SensorData* data) { /* ... */ }

// No copy, cannot be null, and the compiler enforces read-only access
void logReadingByRef(const SensorData &data) { /* ... */ }
```

WHY THIS MATTERS

`const SensorData&` gives the best of both worlds for embedded code: the performance of pass-by-address (no struct copy, no extra stack usage) combined with the safety of pass-by-value (guaranteed non-null, and the compiler will flag any accidental attempt to modify the caller's data inside the function).

5. Structs, Enums, and Unpacking Data

C-Style enum vs. C++ enum class (Scoped Enums)

A traditional C-style enum defines a set of named integer constants, but those names leak directly into the surrounding scope and implicitly convert to plain integers. This means two different enums in the same file cannot reuse a name like `OK` or `ERROR` without a naming collision, and a value from one enum can be silently compared against, or assigned to, a completely unrelated enum or integer without the compiler complaining.

```
// C-style enum – names are NOT scoped, and implicitly convert to int
enum MotorState { IDLE, RUNNING, STALLED };
enum ConnState { IDLE, CONNECTED }; // ERROR: IDLE redeclared

// C++ enum class – scoped, no implicit int conversion
enum class MotorState { Idle, Running, Stalled };
enum class ConnState { Idle, Connected }; // fine, no collision

MotorState m = MotorState::Running;
if (m == MotorState::Stalled) { /* stop the motor */ }
```

enum class (a scoped enum) fixes both problems: every value must be qualified with its enum's name (MotorState::Running, not just Running), so identical member names in different enums never collide, and there is no automatic conversion to int, so a MotorState value cannot be accidentally compared against, or assigned into, a ConnState variable or a raw integer. For a state machine — a motor controller, a connection manager, a protocol parser — this matters directly: state machine bugs caused by comparing or assigning the wrong enum's value are caught at compile time with enum class instead of surfacing as a runtime misbehavior on real hardware.

Struct Padding & Alignment

A struct's total size is usually larger than the simple sum of its members' sizes, because the compiler inserts extra, unused padding bytes between members so that each member starts at a memory address matching its own natural alignment requirement — for example, a 4-byte uint32_t is typically expected to start on an address that is a multiple of 4. Misaligned access is either slower (extra CPU cycles to reassemble the value) or, on some architectures, simply illegal.

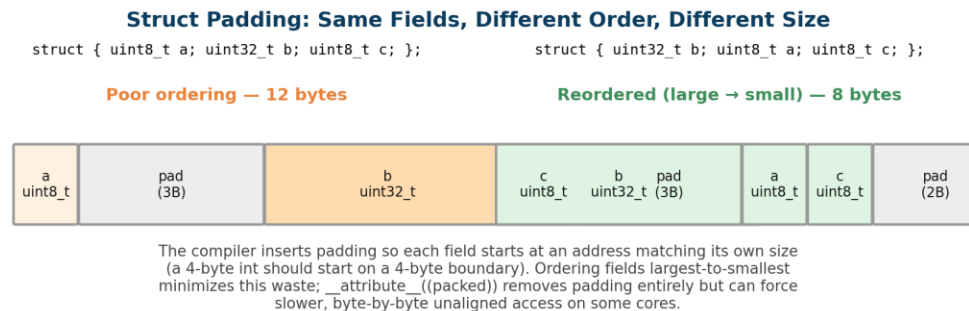


Figure 2.2 — Reordering struct fields from largest to smallest reduces total padding from 12 bytes down to 8 bytes for the same three fields.

Because padding is invisible in the source code, two structs with the exact same fields can end up different sizes purely based on the order those fields are declared in. On a microcontroller with only a few hundred kilobytes of total RAM, an array of thousands of poorly ordered structs can waste a meaningful fraction of available memory on nothing but padding bytes.

- Declaring struct fields from largest type to smallest type (e.g., uint32_t, then uint16_t, then uint8_t fields) is the simplest way to minimize padding without changing behavior.
- `__attribute__((packed))` instructs the compiler to remove all padding entirely, shrinking the struct to the literal sum of its members' sizes.
- Packing is not free: forcing unaligned access on architectures that expect aligned data can make every read/write of a packed field noticeably slower, and on some cores it can trigger a hardware exception. Packed structs are typically reserved for cases where an exact, byte-for-byte memory layout is required —

matching a hardware register map, a communication protocol's wire format, or a fixed on-disk file structure — rather than used by default.

6. Function Pointers & Callbacks

A function pointer is a variable that stores the memory address of a function rather than the address of a piece of data. Because it holds an address, a function pointer can be passed around, stored inside a struct, or reassigned at runtime — and calling through it invokes whichever function it currently points to, rather than a function fixed at compile time.

```
// A function pointer type that matches "void functionName(void)"
typedef void (*CallbackFn)(void);

void onButtonPress(void) { Serial.println("Button pressed!"); }

CallbackFn myCallback = onButtonPress; // store the function's address
myCallback();                          // calls onButtonPress() indirectly
```

Function pointers are the mechanism that makes event-driven, interrupt-based firmware possible. Rather than the main program constantly polling every pin in a loop, a piece of driver code — or the underlying hardware `attachInterrupt()` call — is simply handed the address of a callback function to invoke later, whenever a specific event happens, such as a GPIO pin changing state or a hardware timer expiring.

```
void IRAM_ATTR buttonISR() { buttonPressed = true; }

void setup() {
    // buttonISR's address is registered as the callback for a FALLING edge
    attachInterrupt(BUTTON_PIN, buttonISR, FALLING);
}
```

WHY THIS MATTERS

Function pointers decouple “when something happens” from “what code runs.” The interrupt hardware and the `attachInterrupt()` driver code do not need to know anything about `buttonISR()` at compile time — they only need its address, supplied as a callback, which is exactly the pattern later used throughout FreeRTOS task creation, timer callbacks, and most ESP-IDF driver APIs.

Section 3: C++ OOP & Access Modifiers in Embedded Systems

7. Access Specifiers (public, private, protected)

Encapsulation is the practice of bundling a class's internal data together with the functions that operate on it, and then deliberately hiding the internal representation of that data from any code outside the class. Instead of letting external code reach in and change a value directly, encapsulation forces all interaction to go through a controlled, well-defined set of public functions — the class's interface.

```
class TemperatureSensor {
private:
    uint8_t _sdaPin, _sclPin;    // hardware wiring – not the caller's business
    float _calibrationOffset;    // internal correction value

public:
    TemperatureSensor(uint8_t sda, uint8_t scl) : _sdaPin(sda), _sclPin(scl),
    _calibrationOffset(0.0f) {}
    float readCelsius();         // the only way in/out is through the public API
};
```

Hardware pin numbers and internal sensor calibration values are marked private specifically so that other parts of the firmware cannot set a pin to an invalid or already-in-use value, or overwrite a calibration offset with an unvalidated number, bypassing whatever checks the sensor class's own functions would normally perform. If those values were public, any bug anywhere else in a large codebase could corrupt them directly; keeping them private confines the ways they can change to the small number of functions inside the class itself, which is much easier to review, test, and trust on a device that may be running unattended in the field.

private vs. protected When Inheriting a Base Driver Class

Both private and protected members are hidden from code outside the class, so from an external caller's point of view they look identical. The difference only appears when another class inherits from this one: a protected member remains directly accessible to any derived class, while a private member does not — a subclass can only reach a private member of its base class indirectly, through a public or protected function the base class chooses to expose.

Question	protected	private
Accessible from outside the class?	No	No
Accessible from a derived (subclass) class?	Yes, directly	No — only via the base class's own public/protected functions
Typical embedded use	Fields a derived driver needs to touch directly, e.g. a shared register address in a base HAL class	Internal state the base class must fully control, e.g. a low-level calibration constant no subclass should ever bypass

In practice, a base driver class (for example, a generic Sensor base class) exposes members as protected when it expects specialized subclasses — a TemperatureSensor, a HumiditySensor — to need direct access to shared internals such as an I2C address or a raw register value, and keeps members private when even its own subclasses should not be able to bypass the base class's validation logic.

8. Constructors & Initialization Lists

A constructor is a special member function that runs automatically the moment an object is created, and its job is to put that object into a valid starting state — opening a communication bus, storing pin assignments, or setting default calibration values. A destructor is the mirror-image special function that runs automatically when an object is destroyed, and is responsible for any cleanup the object needs, such as releasing a hardware resource or freeing memory it allocated.

Why Use a Member Initializer List Instead of the Constructor Body

```
class Motor {
    const uint8_t _pin;    // const members MUST be set via an initializer list
    float _maxSpeed;
public:
    // Preferred: member initializer list
    Motor(uint8_t pin, float maxSpeed) : _pin(pin), _maxSpeed(maxSpeed) {}

    // Avoid: assigning in the constructor body instead
    // Motor(uint8_t pin, float maxSpeed) {
    //     _pin = pin;           // ERROR — _pin is const, cannot be assigned after
construction
    //     _maxSpeed = maxSpeed; // this is an ASSIGNMENT, not initialization
    // }
};
```

- Members are constructed before the constructor's body ever runs — assigning a value inside the body is therefore always a second step (construct with a default value, then overwrite it), which does unnecessary extra work for non-trivial member types.
- const members and reference members can only ever be set once, at the moment they are created, so they cannot be assigned inside the constructor body at all — the initializer list is the only place this is legal.
- Members are always initialized in the order they are declared in the class, regardless of the order written in the initializer list, so keeping declaration order and initializer-list order matching avoids compiler warnings and subtle bugs.

WHY THIS MATTERS

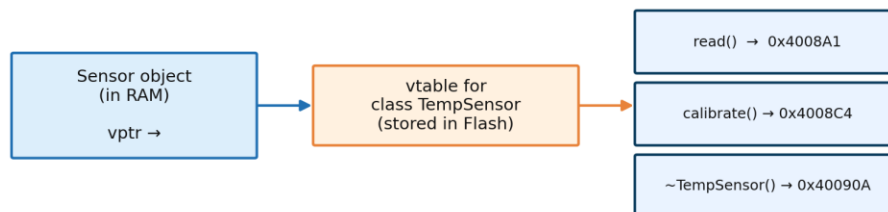
On a microcontroller, avoiding the “construct-then-overwrite” pattern is not just a style preference — for a member that is itself an object with real setup cost (a buffer, a driver object), the initializer list avoids a wasted, redundant initialization step every single time an object of that class is created.

9. Cost of C++ Features in Embedded Systems

Virtual Functions and the vtable

A virtual function lets a base-class pointer or reference call the correct overridden version of a function for whatever derived object it actually points to, decided at runtime rather than at compile time. The mechanism compilers use to make this possible is the virtual table, or vtable: every class that declares at least one virtual function gets one shared vtable — an array of function-pointer addresses, one per virtual function — and every object of that class carries one extra hidden pointer (the vptr) to its class's vtable.

How a Virtual Function Call Resolves at Runtime (vtable)



Every object of a class with virtual functions carries one extra pointer (the vptr) to a shared vtable. A virtual call is one extra memory read (object → vptr → vtable → function) compared to a direct call. On an MCU the bigger cost is usually code/flash size, since the linker must keep every virtual function reachable from the vtable, even unused ones.

Figure 3.1 — A virtual call is resolved through the object's vptr, into its class's vtable, to the correct overridden function address.

Calling a virtual function costs one extra pointer indirection at runtime compared to a normal function call: the generated code follows the object's vptr to its vtable, reads the correct function address out of that table, and only then calls it. On a modern microcontroller clocked in the hundreds of megahertz, this per-call timing overhead is usually small enough to ignore for most application-level code.

The more significant cost on a constrained MCU is memory, in two separate ways. First, every object of a class with virtual functions is larger by one pointer, to hold its vptr. Second, and more importantly, the linker cannot safely remove an unused virtual function the way it can remove an unused ordinary function: because a virtual function's address lives inside a vtable that must be fully resolved, the linker generally has to keep every virtual function of every class actually instantiated in the program, even ones that are never actually called through that vtable at runtime — inflating the compiled firmware's Flash footprint. Some experienced embedded C++ engineers additionally note that on very small microcontrollers, the vtables themselves are copied from Flash into scarce SRAM at startup, consuming RAM the design cannot always spare.

- Runtime cost: one extra pointer dereference per virtual call — usually negligible on a 160–240 MHz core.
- Code-size cost: the linker must keep every virtual function reachable, increasing Flash usage even for unused overrides.
- Object-size cost: every instance carries one extra vptr; this is a one-time cost, shared with any class it inherits from that already has virtual functions.
- Practical guidance: virtual functions are optional in C++, not mandatory as in some other object-oriented languages, precisely so an embedded developer can choose plain, non-virtual functions and static polymorphism where flash size or determinism is tight.

Dynamic Memory Allocation (new / delete / malloc / free)

Dynamic allocation on a general-purpose computer is routine; on a microcontroller it introduces two problems that matter far more than on a desktop or server: non-deterministic timing and heap fragmentation.

- Non-deterministic timing: the time `malloc()` or `new` takes to complete depends on the current state of the heap — how many blocks are free, how they are arranged — not just on the size of the request. In a hard real-time context, such as code running inside FreeRTOS with strict timing deadlines, this unpredictability can cause a task to miss its deadline.
- Heap fragmentation: repeated cycles of allocating and freeing different-sized blocks leave the heap divided into many small, non-contiguous free regions over time. A later allocation request can fail — returning a null pointer — even when the total free memory is more than enough, simply because no single free block is large enough to satisfy it.
- Stack vs. Heap: the stack, by contrast, is deterministic and fragmentation-free by construction — memory is reclaimed automatically and in strict last-in-first-out order the instant a function returns, which is why local variables and function parameters are cheap and predictable, while heap allocations are not.
- Common embedded mitigation: many production firmware projects avoid runtime `malloc()/free()` entirely after startup, preferring statically sized buffers, fixed-size memory pools, or RTOS-provided block allocators, all of which eliminate fragmentation and keep allocation time constant and predictable.

WHY THIS MATTERS

A memory allocation failure that would simply slow down a desktop application can, on an embedded device with only a few hundred kilobytes of RAM and no operator watching a screen, silently corrupt a sensor reading, crash a control loop, or hang a device that then has to be power-cycled in the field. This is precisely why the ESP-IDF and FreeRTOS documentation encourage allocating buffers once at startup and reusing them, rather than allocating and freeing memory continuously inside the main control loop.

References

- <https://developer.espressif.com/blog/2024/08/esp32-memory-map-101/>
- <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/memory-types.html#memory-types>
- <https://developer.espressif.com/blog/esp32-programmers-memory-model/>
- <https://www.youtube.com/watch?v=2htbIR2QpaM&t=5s>
- <https://www.youtube.com/watch?v=nH4dVldgN9Q>
- <https://stackoverflow.com/questions/184537/in-what-cases-do-i-use-malloc-and-or-new>
- <https://www.youtube.com/watch?v=hylEUCIVhQQ>
- https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/mem_alloc.html
- <https://www.pleasedontcode.com/blog/optimizing-ram-usage-in-embedded-systems-with-cc>
- <https://www.geeksforgeeks.org/c/difference-between-static-and-dynamic-memory-allocation-in-c/>
- <https://medium.com/@deepugeorge2007travel/static-vs-dynamic-memory-allocation-d078088262fd>
- <https://www.youtube.com/watch?v=6tIWFEzzx9I>
- <https://www.youtube.com/watch?v=W3pFxBkeJ8>
- <https://www.youtube.com/watch?v=PhjBnhBZrSc>
- <https://esp32.com/viewtopic.php?t=4978>
- <https://dev.to/jayaprakash20/pointers-vs-references-a-deep-dive-into-c-fundamentals-4024>
- <https://medium.com/@matiliththeory/understanding-pointers-vs-references-with-memory-diagrams-039dc74e4117>
- <https://www.geeksforgeeks.org/cpp/enumeration-in-cpp/>
- <https://madhawapolkotuwa.medium.com/mastering-enums-in-c-from-basic-to-advanced-concepts-25c3cdf892b>
- <https://alexanderobregon.substack.com/p/enums-and-typedef-in-c-explained>
- <https://medium.com/@Farhan11637/what-is-structure-padding-aaed25e30080>
- <https://medium.com/@codewithcoders96/understanding-function-pointers-a-beginners-guide-ff74f3f861d9>
- <https://www.geeksforgeeks.org/c/function-pointer-in-c/>
- <https://www.geeksforgeeks.org/cpp/encapsulation-in-cpp/>
- <https://medium.com/@lukaguledani123/encapsulation-in-c-541b2674e025>
- <https://www.pass4sure.com/blog/understanding-encapsulation-in-c/>
- <https://www.geeksforgeeks.org/cpp/access-modifiers-in-c/>
- <https://devabdilah.medium.com/%EF%B8%8F-c-access-specifiers-private-public-and-protected-explained-c007cb9a1f40>
- <https://www.geeksforgeeks.org/cpp/difference-between-constructor-and-destructor-in-c/>
- <https://medium.com/@ankit.adhikari/facing-the-oops-boss-part-1-1-construction-of-destruction-a8254052382c>
- <https://www.geeksforgeeks.org/cpp/vtable-and-vptr-in-cpp/>
- <https://medium.com/@satyadirisala/demystifying-dynamic-dispatch-a-deep-dive-into-virtual-functions-vptr-and-vtable-9574c1ad9bed>
- <https://www.linkedin.com/pulse/virtual-functions-tables-vtable-pointers-vptr-shrikant-shinde-axxwf/>